DESIGN DOCUMENT

EV Public Charging Session Billing and Wallet Engine

Problem Statement: ECE-025

Team ID: E008

Programming Language: Java (Java 21 environment)

Prepared from the project requirements shared by the team. Any interpretation or implementation choice that is not explicitly specified is marked for confirmation.

Version 1.0 | 26 October 2026

Document Control

Item	Details
Project	EV Public Charging Session Billing and Wallet Engine
Problem statement	ECE-025
Team ID	E008
Language	Java; installed runtime reported as Java 21
Document version	1.0 — draft for review
Purpose	Describe the requirements, billing rules, proposed design, and verification plan before completing implementat

Important: this is a draft prepared from the requirements shared by the team. It should be checked against the official submission format if one is later provided.

1. Introduction

Public electric-vehicle (EV) charging stations need a reliable way to record charging sessions, calculate fees, apply membership discounts and taxes, update user wallets, and produce reports. This project proposes a command-line, file-driven Java application that processes charger, user, and session data and generates invoices and account reports.

The system is designed around explicit tariff rules and testable calculations. It does not require a graphical user interface or a web framework.

2. Problem Statement

A charging operator needs to calculate the correct amount for each charging session based on the charger type, energy delivered, charging duration, idle time, membership tier, GST, and minimum invoice rule. The system must also manage wallet top-ups and balances, identify users who become blocked after an unpaid balance, and report revenue by charger for a day.

Manual calculation or inconsistent handling of edge cases can cause incorrect invoices. The proposed engine centralizes the rules and records transactions so that results can be checked.

3. Objectives

- Register chargers with an ID, charger type, and location.
- Register users with an ID, membership tier, and wallet balance.
- Read charging sessions from a CSV input file.
- Calculate energy, time, idle, discount, GST, and final invoice amounts.
- Apply wallet debit, top-up, bonus credit, and ledger updates.
- Block users when the final debit leaves their balance below zero; allow recovery through top-up.
- Generate daily revenue per charger and a blocked-user report.
- Test the specified boundary conditions and error cases.

4. Scope

4.1 In scope

- Command-line Java application; no UI or framework is required.
- File-driven inputs for chargers, users, and charging sessions.
- Billing calculations and invoice output.
- Wallet and transaction ledger processing.
- Daily charger revenue and blocked-user reports.
- Validation and repeatable test cases.

4.2 Out of scope unless later required

- Live charger communication, payment gateway integration, maps, and mobile application.
- Real bank transfers or actual payment processing.
- Cloud hosting or a database; the shared requirements permit a command-line/file-driven solution.

5. Functional Requirements

ID	Requirement
FR-01	Register chargers with unique charger ID, type (AC_7KW, DC_30KW, DC_60KW), and location.
FR-02	Register users with user ID, membership tier, and wallet balance.
FR-03	Read session records containing session ID, user ID, charger ID, plug-in time, charge-complete time, unplug time, kWh delivered, and f
FR-04	Calculate the session invoice using the defined tariff and fee rules.
FR-05	Debit the invoice total from the user wallet and record the transaction and running balance.
FR-06	Accept wallet top-ups of at least ■100; reject smaller top-ups; add a 3% bonus for top-ups of ■1,000 or more.
FR-07	Permit session start only when the wallet balance is at least ■100.
FR-08	Allow a final invoice to take the wallet below zero; mark the user BLOCKED until a top-up brings the balance to at least ■0.
FR-09	Produce daily revenue per charger.
FR-10	Produce a list/report of blocked users.
FR-11	Validate records and report malformed or inconsistent input rather than silently accepting it.

6. Business Rules and Billing Formula

6.1 Energy tariff

Charger type	Energy rate
AC_7KW	■12 per kWh
DC_30KW	■18 per kWh
DC_60KW	■22 per kWh

Energy charge = energy delivered (kWh) × tariff for charger type.

6.2 Time charge

For DC chargers, calculate the duration from plug-in time to charge-complete time. The first 60 minutes have no time charge. Each minute beyond 60 minutes costs ■1. AC_7KW has no time charge.

DC time charge = max(0, charging duration in minutes − 60) × ■1.

Boundary: exactly 60 minutes produces ■0 time charge.

6.3 Idle fee

The first 10 minutes after charge completion are free. After that, charge ■2 per minute until unplug, with a maximum of ■300 per session.

Idle fee = min(max(0, idle minutes − 10) × ■2, ■300).

Boundary: unplugging exactly 10 minutes after charge completion produces ■0 idle fee.

6.4 Membership discount

Membership tier	Discount
SILVER	5% of energy charge only
GOLD	10% of energy charge only
Other/no discount tier	0%

Discount is applied to the energy charge only, not to time charges or idle fees.

6.5 GST and minimum session charge

Taxable amount = energy charge + time charge + idle fee − membership discount.

GST = taxable amount × 18%.

Invoice total = max(■20, taxable amount + GST).

The minimum charge is applied after GST. The exact rounding convention (for example, rounding each component or only the final amount to two decimal places) was not specified. This draft recommends using decimal-safe money arithmetic and rounding the final invoice to two decimal places; confirm if the project owner provides a different rule.

6.6 Faulted sessions

If a session is faulted and energy delivered is less than 0.5 kWh, the session is free. Any amount held for that session must be refunded. A faulted session delivering 0.6 kWh is billable.

The exact meaning and implementation of a 'held amount' was not fully described in the shared requirements. For a file-driven implementation, the design should record any hold/refund explicitly if such a hold exists; otherwise, a zero invoice is applied for the specified free-session condition.

7. Wallet and User Status Rules

- A new session may start only if the user's wallet balance is at least ■100. A balance of exactly ■100 qualifies.
- When a session invoice is finalized, debit the final invoice from the wallet.
- The requirements explicitly allow the wallet to become negative if the final invoice exceeds the balance.
- If the post-debit balance is below ■0, set user status to BLOCKED.
- A blocked user remains blocked until a top-up brings the balance to at least ■0; after that, the user can be unblocked.
- Reject top-ups below ■100.
- For a top-up of ■1,000 or more, add a bonus equal to 3% of the top-up amount. Record both the paid top-up and bonus credit in the ledger.

• Every wallet change should produce a ledger entry containing timestamp or event time, transaction type, amount, and running balance.

7.1 Example top-up

If a user adds ■1,000, the bonus is ■30 and the wallet receives ■1,030 in total credit, assuming the bonus applies to the top-up amount as described. A ■99 top-up is rejected.

8. Input Data Design

8.1 Charger records

Required fields: chargerId, chargerType, location. Charger IDs should be unique.

8.2 User records

Required fields: userId, membershipTier, walletBalance. User IDs should be unique. Status is maintained as ACTIVE or BLOCKED.

8.3 Session CSV

Required columns, in the order specified by the project brief:

```
sessionId,userId,chargerId,plugInTime,chargeCompleteTime,unplugTime,energyDelivered,faultFlag
S001,U001,C001,2026-10-26T10:00,2026-10-26T11:10,2026-10-26T11:25,12.5,false
```

The sample row is illustrative only. The timestamp format and CSV column order should be kept consistent throughout the implementation. Use a documented date-time format such as ISO local date-time unless an official sample file specifies another format.

Input validation should check required fields, known user and charger IDs, valid charger type, parseable timestamps, non-negative energy, and chronological order: plug-in ≤ charge-complete ≤ unplug. Invalid records should be reported and not silently billed.

9. Proposed System Architecture

The application can be organized into small Java classes. The initial files already created are a starting point; further classes can be added after the design is reviewed.

Component	Responsibility
Main	Loads input files, coordinates processing, and writes outputs.
Charger	Stores charger ID, type, and location.
User	Stores user ID, membership tier, wallet balance, and status.
Session	Stores session fields and timestamps.
BillingService	Calculates energy charge, time charge, idle fee, discount, GST, and invoice total.
WalletService (planned)	Validates top-ups, applies bonus credits, debits invoices, and updates blocked status.
LedgerEntry (planned)	Represents each wallet transaction and its running balance.
ReportService (planned)	Produces daily revenue per charger and blocked-user report.
CsvReader/Writer (planned)	Reads input CSV files and writes invoices/reports.

9.1 Processing flow

```
Load charger and user records
  |
Read a session record from CSV
  |
Validate record, user, charger, and timestamps
  |
Is user eligible to start? (balance >= ■100 and not blocked)
  | No -----------------> Record rejection / continue
  | Yes
  v
Calculate energy + time + idle fees
  |
Apply membership discount, GST, and minimum ■20 rule
  |
Apply faulted-session exemption if faulted and kWh < 0.5
  |
Debit wallet; add ledger entry; update BLOCKED status if balance < ■0
  |
Write invoice and update revenue totals
  |
More sessions? -- Yes --> Read next session
  |
  No
  v
Write daily revenue and blocked-user reports
  |
END
```

Note: the exact order of checking session eligibility against historical sessions, and whether an input file contains events in chronological order, must be documented in the implementation. The design assumes sessions are processed in a deterministic order.

10. Invoice Output Design

Field	Meaning
sessionId	Unique charging session identifier
userId / chargerId	User and charger associated with the session
energyCharge	kWh × charger tariff
timeCharge	DC charge for time beyond 60 minutes; zero for AC
idleFee	Fee after 10 free idle minutes, capped at ■300
discount	SILVER/GOLD discount on energy charge only
taxableAmount	Energy + time + idle – discount
gst	18% of taxable amount
invoiceTotal	After GST and minimum ■20 rule, except exempt faulted sessions
walletBalanceAfter	Balance after applying invoice debit
userStatusAfter	ACTIVE or BLOCKED

11. Reporting Requirements

11.1 Daily revenue per charger

For a given business date, group finalized invoices by charger ID and sum the applicable billed amounts. The exact definition of 'revenue'—gross invoice totals, amounts actually collected, or totals excluding

refunded/exempt sessions—was not fully specified. This design recommends reporting finalized invoice totals and documenting how refunds and exempt sessions are handled.

11.2 Blocked users report

List each user whose wallet balance is below ■0 after invoice processing and whose status is BLOCKED. After a top-up raises the balance to at least ■0, the user should no longer appear as blocked if status is updated according to the stated rule.

12. Test Plan

Test ID	Scenario	Expected result
T01	DC charging duration exactly 60 minutes	Time charge = ■0.
T02	Unplug exactly 10 minutes after charge completion	Idle fee = ■0.
T03	Idle time large enough to exceed the cap	Idle fee must not exceed ■300.
T04	Wallet balance exactly ■100 at session start	Session start is permitted if user is not blocked.
T05	Faulted session delivers 0.4 kWh	Session invoice is ■0; held amount, if any, is refunded.
T06	Faulted session delivers 0.6 kWh	Session is billable under normal billing rules.
T07	SILVER membership	5% discount on energy charge only.
T08	GOLD membership	10% discount on energy charge only.
T09	Top-up of ■99	Rejected.
T10	Top-up of ■1,000	■30 bonus; total credit ■1,030.
T11	Invoice debit makes balance negative	User becomes BLOCKED.
T12	Blocked user tops up until balance is at least ■0	User becomes eligible to be unblocked.
T13	Unknown charger or user ID in session	Record is rejected/reported; no silent billing.
T14	Invalid timestamp order	Record is rejected/reported.
T15	AC_7KW session	No time charge, regardless of duration, per given rule.

13. Open Questions and Explicit Assumptions

The shared requirements do not settle every implementation detail. The following points should be handled consistently and recorded in the decision log:

- Money rounding: recommended two-decimal final invoice rounding; confirm whether each component is rounded separately.
- Revenue definition: decide whether daily revenue means invoice totals or money actually collected, and how refunds are represented.
- Fault exemption: apply the stated free rule when faulted and kWh < 0.5; define how held amounts are tracked if the system uses holds.
- Session order: process records in a stable, documented order; if order matters for wallet eligibility, sort by plug-in time or reject out-of-order records as decided.
- Duplicate session IDs: recommended reject duplicate IDs to avoid double billing.
- Minimum charge and fault exemption: the explicit fault exemption should override the ■20 minimum for faulted sessions under 0.5 kWh.

- Time calculations: use whole completed minutes unless an official example defines another method; test partial-minute boundaries.
- Blocked user: do not allow a blocked user to start another session until top-up brings balance to at least ■0 and status is restored.

14. Implementation Plan

Stage	Work
1. Design	Review requirements, finalize formulas, assumptions, architecture, and tests.
2. Data models	Complete Charger, User, Session, Invoice, and ledger-related classes.
3. Billing	Implement decimal-safe calculations and fault exemption.
4. Wallet	Implement session eligibility, top-up, bonus, debit, ledger, and blocked status.
5. File processing	Read charger/user/session files and write invoices/reports.
6. Testing	Automate required edge-case tests and record actual results.
7. Submission	Complete README, design PDF, test suite, decision log, and demo recording as required.

15. Completion Checklist

- Design document reviewed against the original project brief.
- All tariff and billing formulas implemented.
- Faulted-session exemption overrides normal minimum charge when applicable.
- Wallet top-up, bonus, debit, ledger, and blocked status work correctly.
- CSV parsing and input validation are implemented.
- Daily revenue and blocked-user reports are generated.
- All required edge-case tests pass and results are documented.
- README explains how to compile and run the project.
- Repository includes the required design, tests, decision log, and demo.

End of design document.